MDB200

Finding Slow Operations with Logs and Metrics

Release: 20260220

Topics we cover

Database Performance Monitoring

Database Logs in MongoDB

Log File Format

Database Profiler

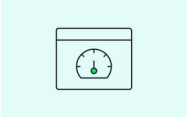
System and Database Monitoring

Causes of Slow Operations

Database Performance Monitoring

Profiling, slow query analysis, and system monitoring: Analyze query performance and identify slow operations with Database logs, profiling and other monitoring tools.

Optimizing indexes: Apply indexing strategies to improve query performance and reduce latency.



Optimize resource utilization: Configure caching mechanisms to enhance read efficiency, reduce database load and set alerts to flag any memory thresholds.

Logging

MongoDB maintains a log of events, including entries such as incoming connections, commands run, and issues encountered

Useful for :

- Diagnosing issues related to slow queries
- Monitoring your deployment
- Tuning performance

Accessing Database Logs

The Database log file is stored as a text file on the server host

If hosted in Atlas, use the GUI/API to download

Access the last 1024 lines of the log via getLog

Note: You can also push Database logs in an AWS S3 bucket using Push-Based Log Export

File System	/var/log/mongodb/mongod.log
Hosted in Atlas	Download from the UI or with API (or from the AWS S3 bucket if using Push-Based Log Export*)
Database	Access with the **getLog** command

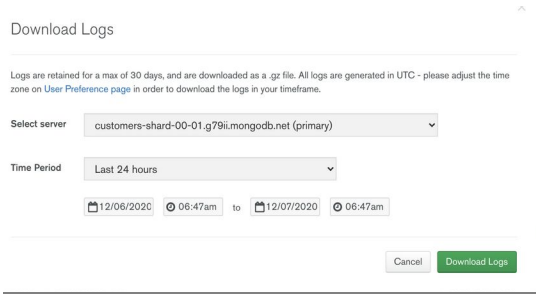
The default location of the log is /var/log/mongodb/mongodb.log; however, this often varies depending on how MongoDB has been configured.

*Push-Based Log Exports allow you to configure Atlas M10+ to automatically export database logs to your AWS S3 bucket via IAM role.

Accessing Database Logs in Atlas

In the Cluster page:

1. Select your cluster
2. Click the ellipsis icon (...) for to the cluster containing the mongod instance whose logs you want to download
3. Select Download Logs
4. Select host and time period



Note: Database Log is **not available** for download from shared tier servers (M0 or Flex). It can be accessed using the getLog command (show log global command) but only has information specific to you - not general server information.

getLog command

The getLog command gets the last 1024 log entries

Returns a single document where **logs** are in an array of JSON documents

```
> var loglines = db.adminCommand({getLog:"global"})

> printjson(loglines)

{
        totalLinesWritten : 2625918,
        log : [
                        <LOG LINES HERE>
        ],
        ok : 1,
        $clusterTime : {
                clusterTime: Timestamp({ t: 1755508582, i: 1 }),
                signature: {
                ...
    }
  },
  operationTime: Timestamp({ t: 1755508582, i: 1 })
}
```


- It does not read data from the mongod log file. It reads data from the RAM cache of logged mongod events.
- It returns log data in escaped Relaxed Extended JSON v2.0 format.
- totalLinesWritten is the size of the full on-disk logfile

Copying logs to a collection

getLog returns JSON data

Easy to insert into a collection

Use find() and aggregate() to examine log data

```
> use temp
> var x = db.adminCommand( { getLog: "global" } )
> db.log.drop()
> db.log.insertMany(x.log.map(d=>JSON.parse(  d.replace(/\$/g,"_"))))
> db.log.find({}).pretty()
{
        _id : ObjectId("5fcf704fda3f8a47fffdc6e5"),
        t : {
                $date : "2020-12-08T11:52:56.311+00:00"
        },
        s : "I",
        c : "NETWORK",
        id : 22944,
        ctx : "conn329",
        msg : "Connection ended",
        attr : {
                remote : "192.168.254.30:55800",
                connectionId : 329,
                connectionCount : 36
        }
  }
```



```
use temp
var x = db.adminCommand( { getLog: "global" } )
db.log.drop()

db.log.insertMany(x.log.map(l=>JSON.parse(l)))
db.log.find({l:/COLLSCAN/}).pretty()
```

We can also load a log file into mongodb as a collection using mongoimport.

Querying logs in a collection

Query your log collection using find()

Use aggregations like **$sortByCount** to group and analyse log entries

```
> use temp

> db.log.find({c:"ACCESS",msg:/^Successful/},
{_id:0,"attr.client":1}).pretty()
  { attr : { client : "192.168.248.14:52798" } }
  { attr : { client : "192.168.248.14:52808" } }
  { attr : { client : "192.168.248.14:52824" } }
  { attr : { client : "192.168.248.14:52838" } }
  { attr : { client : "192.168.248.14:52844" } }
  { attr : { client : "192.168.248.14:52860" } }
  { attr : { client : "192.168.248.14:52876" } }

> db.log.aggregate([{$sortByCount:"$c"}])
  { _id : "NETWORK", count : 728 }
  { _id : "ACCESS", count : 176 }
  { _id : "SHARDING", count : 105 }
  { _id : "CONNPOOL", count : 12 }
  { _id : "QUERY", count : 1 }
```


- To view the log in the shell just type **show log global**
- If the above query doesn't return result, try the following query:

```
db.log.find( { c:"ACCESS", msg:"Authentication succeeded" },
{ _id:0, "attr.remote":1 } ).pretty()
```

Finding slow operations using log

Queries appear in the log if their duration exceeds the default slow operation threshold (100ms)

Can see how long they took, their query plan, and many other values

```
> use temp
> query = {"attr.ns":{$ne :"local.oplog.rs"},c:"COMMAND"}
> bytime = {"attr.durationMillis":-1}
> db.log.find(query).sort(bytime).pretty()
```

Log file output format

Severity

- F: Fatal
- E: Error
- W: Warning
- I: Informational

The attr section is the most important to query but varies

Truncated shows if the log entry was too large

```
{
  t: <Datetime>, // timestamp
  s: <String>, // severity
  c: <String>, // component
  ctx: <String>, // context
  id: <String>, // unique identifier
  msg: <String>, // message body
  attr: <Object> // additional attributes
(optional)
  tags: <Array of strings> // tags (optional)
  truncated: <Object> // truncation info (if
truncated)
  size: <Integer> // original size of entry (if
truncated)
}
```

Log file output format

Components let you filter specific areas of functionality or messages

ACCESS	Messages related to access control, such as authentication
COMMAND	Messages related to database commands such as count
WRITE	Messages related to the write operations, such as update commands
INDEX	Messages related to indexing operations, such as creating indexes
JOURNAL	Messages related specifically to storage journaling activities
NETWORK	Messages related to network activities, such as accepting connections
QUERY	Messages related to queries, including query planner activities

Full set of components are - CONTROL, ELECTION, FTDC, GEO, INDEX, INITSYNC, JOURNAL, NETWORK, QUERY, RECOVERY, REPL, REPL_HB, ROLLBACK, SHARDING, STORAGE, TXN, WRITE, ACCESS

Database Profiler

Database profiler records information about slow operations:

- Definition of slow is configurable and can mean all operations
- The size of profiling data is capped at 10MB
- Recorded in a collection called system.profile
- Can be enabled globally or for any database
- Creates extra writes
  - Enabling profiler for every operation, creates extra write
  - Always turn off Profiling unless required - Affects performance

- The database profiler collects detailed information about Database commands executed against a running MongoD instance.
- This includes CRUD operations as well as configuration and administration commands.
- The profiler writes all the data it collects to the system.profile collection, a capped collection of 10 MB in the database being profiled
- The profiler is off by default, and you can enable the profiler on a per-database or per-instance basis

The setProfilingLevel() command

`db.setProfilingLevel(level, slowms)`

level: 0, 1, or 2

0 - Profiler is off

1 - Store only slow queries (default)

2 - Store all queries*

*Not recommended in Production

slowms: threshold in milliseconds for slow operations (default it is 100)

The queries are stored in a collection `system.profile`.

Check the Profile settings with `db.getProfilingStatus()`

```
> db.setProfilingLevel(1,5)
  {"was" : 0,"slowms" : 100,"sampleRate" : 1,"ok" : 1, ... }
> db.getProfilingStatus()
  { "was" : 1, "slowms" : 5, ... }
> use sample_airbnb
> db.listingsAndReviews.find({amenities:"Snooker"})
> use admin
> var loglines = db.runCommand({getLog:"global"})
> printjson(loglines)
...
"2021-07-09T11:49:00.237+0000 I  COMMAND  [conn214] command
sample_airbnb.listingsAndReviews appName: \"MongoDB Shell\" command:
find { find: \"listingsAndReviews\", filter: { amenities:
\"Snooker\" }, lsid: { id: UUID(\"6496eb58-cea3-4c05-b237-
0d04e2ffc60b\") }, $clusterTime: { clusterTime: Timestamp(1594295320,
1), signature: { hash: BinData(0,
E2DB7AAFBFE43F8A424F60EA04033378C1B73B89), keyId: 6847444150736912387 }
}, $db: \"sample_airbnb\" } planSummary: COLLSCAN keysExamined:0
docsExamined:5555 cursorExhausted:1 numYields:43 nreturned:0
queryHash:0AEEE9D2 planCacheKey:0AEEE9D2 reslen:246 locks:
{ ReplicationStateTransition: { acquireCount: { w: 44 } }, Global:
{ acquireCount: { r: 44 } }, Database: { acquireCount: { r: 44 } },
...
```


The method `db.setProfilingLevel(level, <options>)` returns a document that contains the previous values of the settings.
Note: This command is not allowed on Atlas Shared Tiers.

You can also specify other parameters in addition to the slowms
- sampleRate (default 0.1) - between 0 and 1, represents the proportion % of queries to log
- A filter to determine what operations are logged

```
db.setProfilingLevel(level, slowms, samplerate, filter)
```

Examples:
`db.setProfilingLevel(1, {slowms: 150, sampleRate: 0.1 })`
This is the same as
`db.setProfilingLevel(1, 150)`

The db profiler can also be used to find slow queries - slowms sets the threshold for 'slow'

Query Profiler Data

The **`show profile`** helper in the mongo shell displays the five most recent operations that match the **setProfilingLevel** filters.

We can also query or aggregate the **system.profile** collection in any database

```
> show profile
query sample_airbnb.listingsAndReviews 5ms Wed Dec 09
2024 16:39:04
command:{
      find : "listingsAndReviews",
      limit : NumberLong(21),
      maxTimeMS : NumberLong(15000),
      skip : NumberLong(0),
      $db : sample_airbnb,
      $readPreference : {
            mode : "primaryPreferred"
      }
}
keysExamined:0
docsExamined:21
cursorExhausted
numYield:0
nreturned:21
locks:{
      ReplicationStateTransition : { ...


> //Find the slowest ops
> db.system.profile.find().sort( { millis : -1 })
```


use sample_airbnb

And run above commands

Tools for System Monitoring

```
> mongotop

2019-04-29T15:35:27.785-0400 connected to: 127.0.0.1
                    ns    total    read   write
     admin.system.roles      0ms      0ms      0ms
     admin.system.users      0ms      0ms      0ms
  admin.system.version      0ms      0ms      0ms
config.system.sessions      0ms      0ms      0ms
```

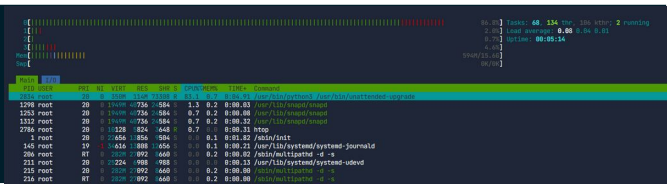
mongotop monitors the **time MDB spends reading from and writing to** collections

```
> mongostat -u user --authenticationDatabase admin

insert query update delete getmore command dirty used flushes vsize  res qrw arw net_in net_out conn
    time
   991     *0     *0      0     2|0  3.4% 4.5%       0 2.90G 297M 0|0 0|0  12.9m   84.2k    2 Oct
6 09:45:37.478
   989     *0     *0      0     2|0  3.6% 4.7%       0 2.91G 310M 0|0 0|0  12.9m   84.1k    2 Oct
6 09:45:38.476
   988     *0     *0      0     1|0  3.7% 4.8%       0 2.92G 323M 0|0 0|0  12.8m   83.8k    2 Oct
6 09:45:39.481
   976     *0     *0      0     2|0  3.9% 5.0%       0 2.94G 335M 0|0 0|0  12.7m   83.7k    2 Oct
6 09:45:40.476
```

mongostat shows **real-time statistics** about operations performed by instances in a cluster

```
> htop
```



htop shows overall **system performance** (CPU, memory, disk usage, and network activity) in real time

mongotop provides insight into collection-specific operations, which can be used to pinpoint resource bottlenecks and optimize read/write workloads.

mongostat shows metrics such as insert, query, update, delete, and command rates. Additionally, it logs system resource usage like memory (vsize and res), connection counts (conn), network traffic (net_in and net_out), and replica set state (repl). This command is primarily used to monitor database activity and diagnose potential performance issues.

htop displays real-time information about CPU usage, memory, disk activity, network usage, and running processes in an easily navigable interface. htop and top are usually pre-installed in most Linux distributions. Other tops like atop or btop may require manual installation.

db.serverStatus()

Returns a document that provides an overview of the state of the MongoDB instance (mongod or mongos)

Monitoring applications can run this command at a regular interval to collect statistics about the instance

Includes up to 1,500 parameters

```
> db.serverStatus()

{
        host : "atlas-10yszs-shard-00-
01.g79ii.mongodb.net",
        version : 6.0.6,
        process : "mongod",
        pid : NumberLong(16421),
        uptime : 340215,
        uptimeMillis : NumberLong(340215520),
        uptimeEstimate : NumberLong(340215),
        localTime : ISODate("2024-12-08T06:43:21.953Z"),
        asserts : {
                regular : 0,
                warning : 0,
                msg : 0,
                user : 3576,
                rollovers : 0
        },connections : {
current : 36,
available : 1464,
totalCreated : 41747,
active : 5,
}

      ...
```


The **`db.serverStatus()`** command returns a large amount of data. To return a specific object or field from the output, append the object or field name to the command.

`db.runCommand({ serverStatus: 1}).metrics`
or
`db.serverStatus().metrics`

`db.runCommand({ serverStatus: 1}).metrics.commands`
`db.runCommand({ serverStatus: 1}).metrics.commands.update`

Admin should use **`db.serverStatus()`**

`db.serverStatus()` and **`db.runCommand({ serverStatus: 1})`** both return the same values.

db.serverStatus()

To exclude fields that are not wanted, use the same syntax as projection

You can then see specific parts using the MongoDB shell

```
> db.serverStatus( { repl: 0,  metrics: 0, locks: 0 } )

> db.serverStatus().connections
{
  current: 83,
  available: 1417,
  totalCreated: 1227,
  rejected: 0,
  active: 27,
  threaded: 83,
  limitExempt: 67,
  exhaustIsMaster: 3,
  exhaustHello: 16,
  awaitingTopologyChanges: 19
}


> db.serverStatus().wiredTiger.cache
```

Some causes of slow operations

- Missing indexes - leads to lots of Disk I/O and CPU usage
- Insufficient memory - leads to increase disk operations
- Writes to the Cache outstripping disk write capability
  - Pauses waiting for cache to flush/checkpoint
- Not enough Cache allocated - leads to paging into cache
- Locking - you can see in logs what locks things take
  - Most things don't block others - but a few admin things do
  - One admin task could block production
- Excessive CPU usage
  - Constantly authenticating
  - Extremely large arrays
  - Extremely complex Aggregations
  - Sorting without index
  - Using Server-side JavaScript

These are many of the most common causes of performance problems.
Often they come from bad design or a lack of understanding - fixing most of these is not something an Operations person can do without a developer.

Server-side JavaScript has been deprecated as of MongoDB version 8.0.

Slow Ops output

```
{ op : "query",
  ns : "sample_airbnb.large",
  command : {
    find : large,
    filter : {  amenities : "Snooker" },
    $db : "sample_airbnb" },
    keysExamined : 0,
    docsExamined : 277750,
    numYield : 868,
    nreturned : 0,
    queryHash : "0AEEE9D2",
    planCacheKey : "0AEEE9D2",
    locks : {
      Global : {
        acquireCount : {
          r : NumberLong(870) }},
      Database : {
        acquireCount : {
          r : NumberLong(869) }},
      Collection : {
        acquireCount : {
          r : NumberLong(869) }  },
      storage : {
        data : {
          bytesRead : Long(4744493955),
          timeReadingMicros:Long(14794471)},
          timeWaitingMicros : {
            cache :Long(3256)}},
  responseLength : 233,
  protocol : "op_msg",
  millis : 15640,
  planSummary : "COLLSCAN",
  execStats : {
    stage : "COLLSCAN",
    filter : {
      amenities : {
        $eq : "Snooker" } },
    nReturned : 0,
    executionTimeMillisEstimate : 15227,
    works : 277752,
    advanced : 0,
    needTime : 277751,
    saveState : 868,
    docsExamined : 277750},
  ts : ISODate(2020-12-09T16:41:22.563Z),
  client : 86.179.217.72,
  appName : "MongoDB Shell",
}
```


This is output from the Log OR Profiling . Some fields to consider are:

- ns: namespace
- docExamined: number of documents reviewed in the query
- keyExamined: number of indexed docs reviewed (if using an index)
- nreturned: docs returned by the query
- planSummary: execution planned by MongoDB to fetch the information (COLLSCAN = Collection Scan)
- execStats: Stats on execution times (includes execution time estimates)

A few fields have been omitted.

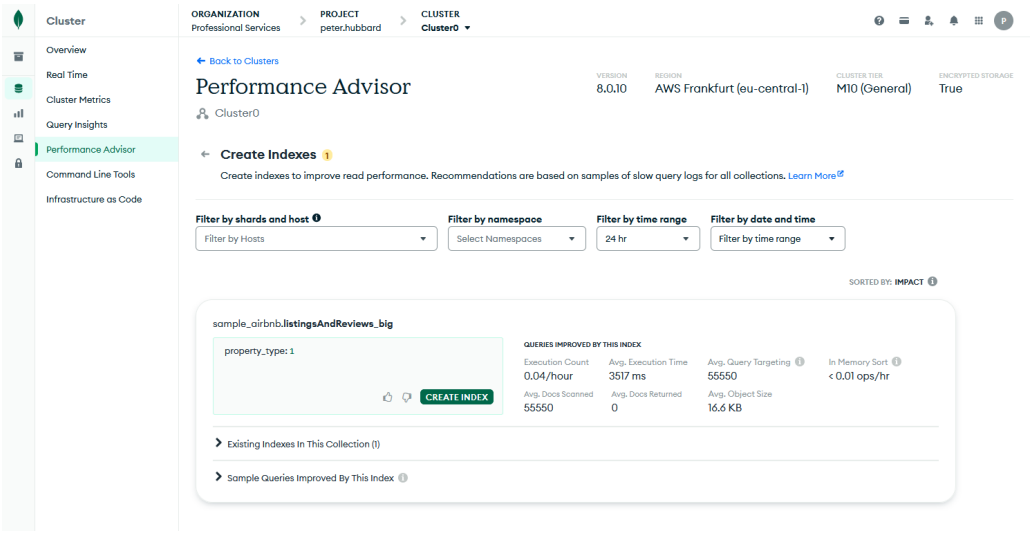
Finding slow operations in Atlas

There are various ways to analyze slow queries in Atlas or Cloud/Ops Manager:

- Performance Advisor
- Real-Time Performance Panel
- Query Profiler

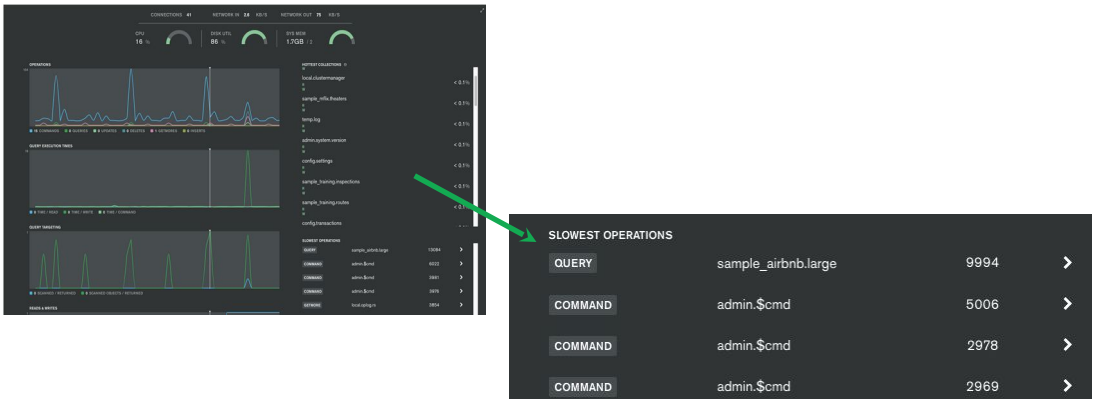
- Only available on M10+ and larger Clusters
- If required, the trainer can show the demo on M10+ Clusters.
- Details: https://docs.atlas.mongodb.com/analyze-slow-queries/

Atlas - Performance Advisor



- Click on the Cluster name.
- Click **Performance Advisor**.
- Check for indexes to create or drop, or schema improvements
- "Create Indexes" recommendations can be filtered by host, namespace and time

Atlas - RTPP / Compass

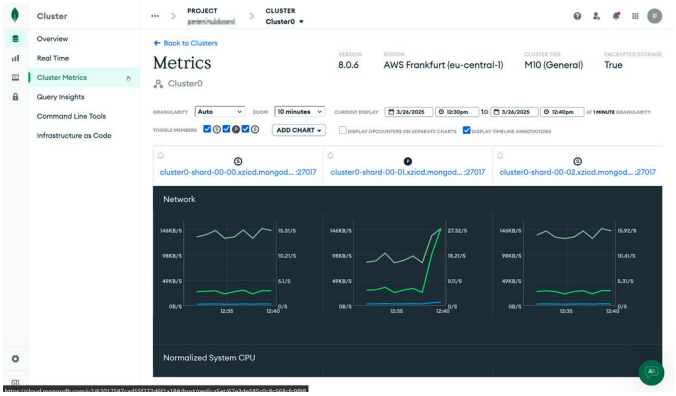


- Real-Time Performance Panel (RTPP)
- Only available on M10 and larger Clusters
- Same functionality available on MongoDB Compass

Atlas - Cluster Metrics

Atlas and Ops/Cloud Manager show metrics over time:

- Ops per second
- Queue lengths
- Scan and Order
- Targeting
- Query Executor
- Document Metrics
- Execution Times
- Network usage
- Cache activity
- Memory usage

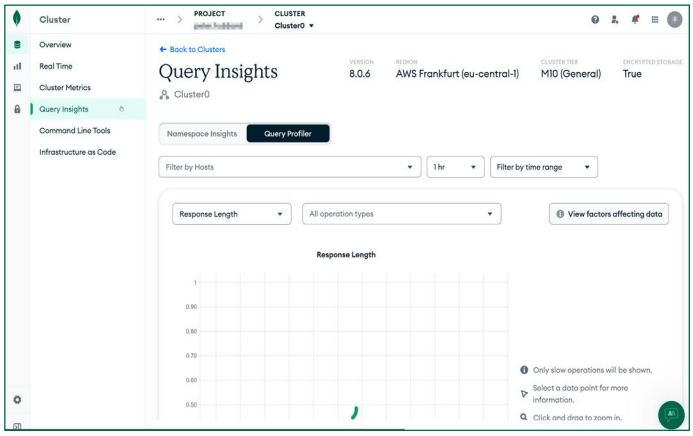


- Full metrics require an M10 or higher or Cloud Manager Standard/Premium
- The free version has more limited information
- This is a great tool for determining longer term problems.

Atlas - Query Insights

Namespace Insights shows query latency per collection

Query Profiler shows slower operations graphically



- Only available on Dedicated (M10+) Clusters
- See https://www.mongodb.com/docs/atlas/analyze-slow-queries/ and https://www.mongodb.com/docs/atlas/namespace-insights/

Quiz Time!

#1. How many log entries does the getLog command return?

A	512	B	1024	C	4096
D	65535	E	24 hours of		

Answer in the next slide.

#1. How many log entries does the getLog command return?

A	512	B	1024	C	4096
D	65535	E	24 hours of		

getLog only shows the most recent 1024 logged mongod events, and is not a replacement for the MongoDB log file.

#2. What does the Data profiler do when is enabled with `setProfilingLevel(1,2)`?

A	Records all write operations	B	Puts operations taking more than 2ms in the profile database	C	Puts operations taking more than 2ms in the logfile
D	Puts all operations in the logfile	E	Causes large write load within the database cache		

Answer in the next slide.

#2. What does the Data profiler do when is enabled with `setProfilingLevel(1,2)`?

A	Records all write operations	B	Puts operations taking more than 2ms in the profile database	C	Puts operations taking more than 2ms in the logfile
D	Puts all operations in the logfile	E	Causes large write load within the database cache		

The setProfilingLevel() takes 2 arguments, the first one sets the level (0/1/2) and the second argument is what we define as slow in ms so here we are looking to record all operations taking longer than 2 ms to be recorded in the profiler, such a tiny threshold will lead to a large write load due to most operations (not all) not finishing that fast. B,C,E are valid.

#3. Which of these are likely causes of excessive CPU usage?

A	Bad schema design with large arrays	B	Use of JavaScript on the server	C	High number of unauthenticated connection requests
D	Large numbers of updates	E	Sorting data without an index		

Answer in the next slide.

#3. Which of these are likely causes of excessive CPU usage?

A Bad schema design with large arrays

B Use of JavaScript on the server

C High number of unauthenticated connection requests

D Large numbers of updates

E Sorting data without an index

Note: Large numbers of updates usually do not cause excessive CPU usage, but there are some edge cases, such as if the update doesn't hit any indexes and causes a collection scan. In the context of this question - if all things are normal in the DB, D would not be a valid choice whereas the others can cause issues.

SKILL BADGE ASSESSMENT – 10 MINUTES

Query Optimization

MongoDB Skill

Query Optimization

Take the Assessment

https://mdb.link/Skill-QueryOptimization-PS

Questions

Ask your neighbor

Raise your hand

Let's do it together!

SKILL BADGE ASSESSMENT – 10 MINUTES

Performance Tools and Techniques

MongoDB Skill

Performance Tools and Techniques

Take the Assessment

https://mdb.link/Skill-PerformanceToolsTechniques-PS

Questions

Ask your neighbor

Raise your hand

Let's do it together!

Recap

Log of events hosted as a file in the Database server

getLog allows us to output the log into a collection and query it

With Atlas the log can be downloaded

Performance Advisor

Real-Time Performance Panel (RTPP)

Query Profiler